

COMPUTER NETWORKS LABORATORY
(CSDC - 0236)

B.Tech IV th Semester
(January – June 2025)

Submitted by :-

Khushal Garg (23103077)
Group-G3

Submitted to :-

Dr. Samayveer Singh



Department of Computer Science & Engineering
Dr. B. R. Ambedkar National Institute of Technology Jalandhar
-144008, Punjab, India

S. No.	Week No.	Name of Practical
1	Week 1	a) Hands-on with Network Physical Components for Designing a LAN Cable and Enabling Data Transmission by Connecting Two Computers. b) Introduction to Cisco Packet Tracer and Implementation of a Small Network.
2	Week 2	Design a network scenario where four distinct departments (IT, HR, Sales, and Finance), each operating within separate local area networks (LANs), are directly connected through intermediate devices. Each department should have at least four end systems and exist within the same overall network. For the given scenario, perform the following tasks: (a) Assign a static IP address to every end device in the network. (b) Label all network devices appropriately. (c) Label each network interface with its assigned IP address. (d) Implement a unique network topology for each department. (e) Demonstrate data transmission between departments. (f) Provide an explanation of the overall network topology.
3	Week 3	Objective 3: Create a scenario where four different departments (IT, HR, Sales, Finance) of different local area networks are directly connected through intermediate devices. Each group has at least 4 end systems. Each topology has at least one router, and each department is connected to a central router. Do the following for the given scenario: a) Provide the Static IP Address to every end

		device available in the network. b) Label each and every interface with their IP Address. c) Use a different topology for each department d) Configure static routing. e) Discuss the concept of the gateway and provide the default gateway address to devices. f) Show the data transmission between each department g) Also, explain the overall topology of the network.
4	Week 4	Objective : Create a scenario where four different departments (IT, HR, Sales, Finance) of different local area networks are directly connected through a central router. Each department has at least 4 end systems. Each topology has at least one router. Do the following for the given scenario: a) Provide the Static IP Address to every end device available in the network. b) Label each and every interface with their IP Address. c) Use a different topology for each department d) Configure RIP routing. e) Show the data transmission between each department f) Also, explain the overall topology of the network.
5	Week 5	Objective : Create a scenario where four different departments (IT, HR, Sales, Finance) of different local area networks are directly connected through a central router. Each department has at least 4 end systems. Each topology has at least one router. Do the following for the given scenario: a) Provide the Static IP Address to every end device available in the network. b) Label each and every interface with their IP Address.

		c) Use a different topology for each department d) Configure OSPF routing. e) Show the data transmission between each department f) Also, explain the overall topology of the network.
6	Week 6	Objective#6: Write a menu-driven program to implement and validate the following error detection/correction techniques: a) Checksum b) Longitudinal Redundancy Check c) Cyclic redundancy check d) Hamming Code

DR. B.R. AMBEDKAR NATIONAL
INSTITUTE OF TECHNOLOGY, JALANDHAR

Week 1



NAME: Khushal Garg

ROLL NO.: 23103077

SECTION: 4A

GROUP: G3

BRANCH: CSE

Objective : a) Hands-on with Network Physical Components for Designing a LAN Cable and Enabling Data Transmission by Connecting Two Computers. b) Introduction to Cisco Packet Tracer and Implementation of a Small Network.

1. Introduction : Designing a LAN cable and establishing data transmission between two computers using network physical components.

2. Required Components

- Two computers with Ethernet ports
- RJ-45 connectors (Male and Female)
- Cat6 Ethernet cable
- Crimping tool

3. Designing a LAN Cable

- **Step 1:** Measure and cut the Ethernet cable to the required length.
- **Step 2:** Strip about 1 inch of the outer insulation from both ends of the cable.
- **Step 3:** Insert the wires into the RJ-45 connectors and ensure they are properly aligned.
- **Step 4:** Use a crimping tool to secure the connectors onto the cable.

4. Connecting Two Computers

- **Step 1:** Plug one end of the LAN cable into the Ethernet port of the first computer.
- **Step 2:** Connect the other end to the second computer.
- **Step 3:** Configure IP addresses.
- **Step 4:** Test connectivity using the ping command or file sharing.

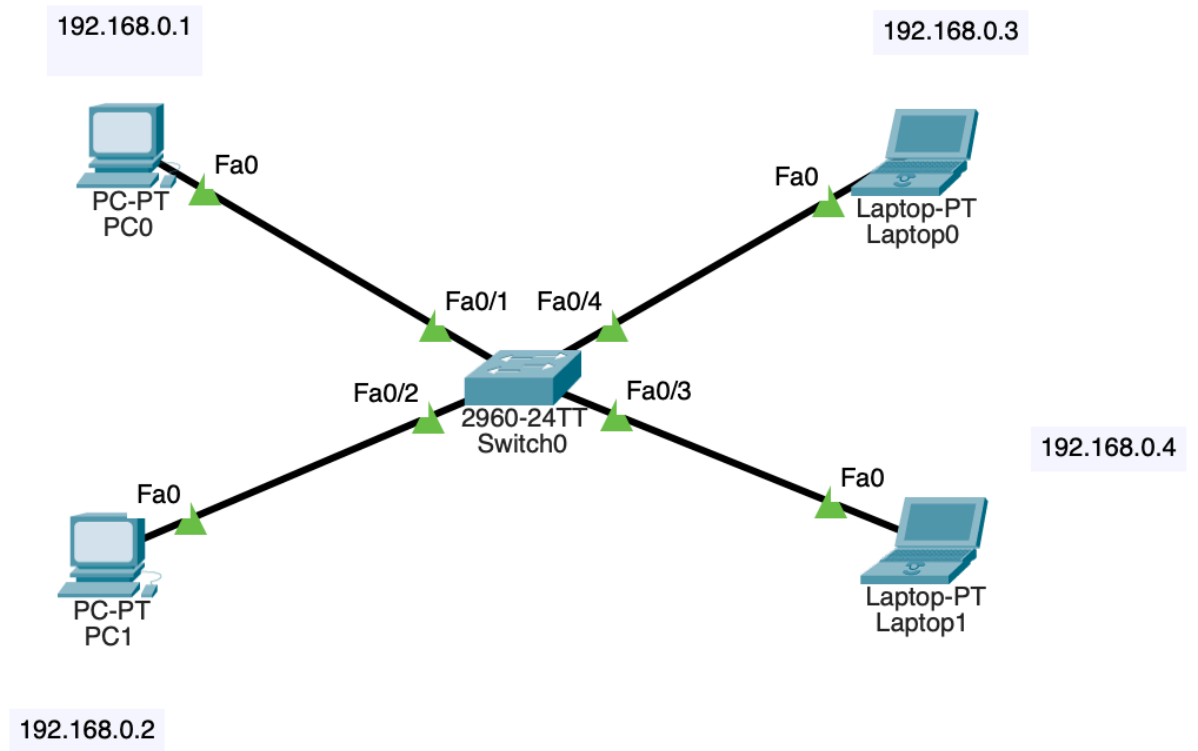
5. Data Transmission Verification

- **Ping Test:** Run “ping<IP>” to check connectivity.
- **File Sharing:** Enable file sharing and transfer a test file between computers.
- **Network Diagnostics:** Use network troubleshooting tools to confirm a stable connection.

Cisco Packet tracer :

IP Addressing Scheme :

- PC0: 192.168.0.1
- PC1: 192.168.0.2
- Laptop0: 192.168.0.3
- Laptop1: 192.168.0.4



Fire	Last Status	Source	Destination	Type	Color	Time(sec)	Periodic	Num	Edit	Delete
	Successful	PC1	Laptop1	IC...	Black	0.000	N	0	(...	(delete)
	Successful	PC0	Laptop0	IC...	Red	0.000	N	1	(...	(delete)

DR. B.R. AMBEDKAR NATIONAL

INSTITUTE OF TECHNOLOGY, JALANDHAR

Week 2



NAME: Khushal Garg

ROLL NO.: 23103077

SECTION: 4A

GROUP: G3

BRANCH: CSE

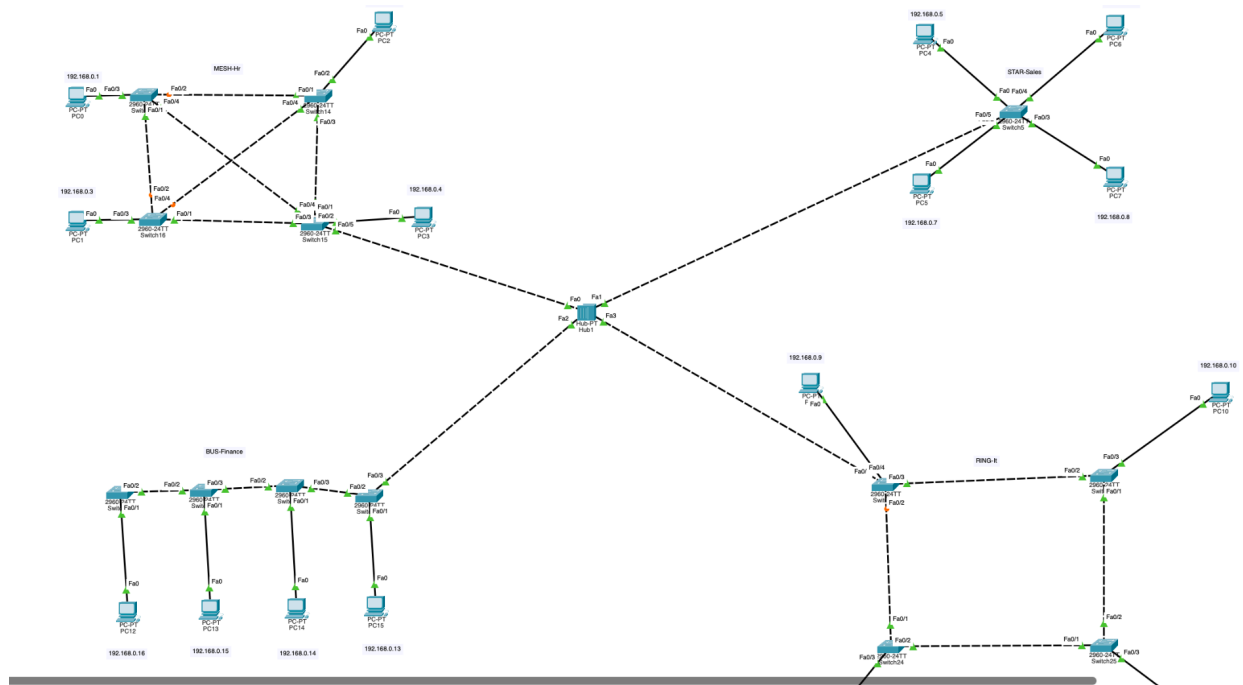
Objective: Design a network scenario where four distinct departments (IT, HR, Sales, and Finance), each operating within separate local area networks (LANs), are directly connected through intermediate devices. Each department should have at least four end systems and exist within the same overall network. For the given scenario, perform the following tasks:

- a) Assign a static IP address to every end device in the network.
- b) Label all network devices appropriately.
- c) Label each network interface with its assigned IP address.
- d) Implement a unique network topology for each department.
- e) Demonstrate data transmission between departments.
- f) Provide an explanation of the overall network topology.

1. IP Addressing Scheme

All end devices are assigned static IP addresses within a structured subnet.

- **Sales Department (192.168.0.1 - 192.168.0.4)**
 - PC0: 192.168.0.1
 - PC1: 192.168.0.2
 - Laptop0: 192.168.0.3
 - Laptop1: 192.168.0.4
- **IT Department (192.168.0.5 - 192.168.0.8)**
 - PC2: 192.168.0.5
 - PC3: 192.168.0.6
 - Laptop2: 192.168.0.7
 - Laptop3: 192.168.0.8
- **HR Department (192.168.0.9 - 192.168.0.12)**
 - PC4: 192.168.0.9
 - PC5: 192.168.0.10
 - Laptop4: 192.168.0.11
 - Laptop5: 192.168.0.12
- **Finance Department (192.168.0.13 - 192.168.0.16)**
 - PC6: 192.168.0.13
 - PC7: 192.168.0.14
 - Laptop6: 192.168.0.15
 - Laptop7: 192.168.0.16



2. Network Devices and Labeling Each department's network devices (switches, hubs) are labeled appropriately in the Packet Tracer file:

- Sales: Switch_Sales
- IT: Switch_IT
- HR: Switch_HR
- Finance: Switch_Finance
- Centralized Hub

3. Network Interface Labeling Each network interface has been labeled with its corresponding IP.

4. Network Topology Each department follows a unique topology to accommodate specific network requirements:

- **IT Department:** Ring topology
- **HR Department:** Bus topology
- **Sales Department:** Star topology
- **Finance Department:** Extended star topology

5. Data Transmission Between Departments

- Inter-departmental communication is established through a core hub.
- Data transmission can be monitored using simulation tools in Cisco Packet Tracer.

Fire	Last Status	Source	Destination	Type	Color	Time(sec)	Periodic	Num	Edit	Delete
	Successful	PC15	PC8	IC...		0.000	N	0	(...	(delete)
	Successful	PC8	PC7	IC...		0.000	N	1	(...	(delete)

6. Explanation of Overall Network Topology This network follows a **hybrid topology**, integrating multiple network topologies to enhance efficiency and resilience. Each department's LAN follows an independent topology, while a core router manages inter-departmental communication.

- **Scalability:** The network allows easy expansion by adding new devices.
- **Redundancy:** The hybrid design ensures minimal failure impact.
- **Security:** Each department operates in a separate subnet, reducing unauthorized access risks.
- **Efficiency:** Traffic management is optimized using network switches and hubs.

DR. B.R. AMBEDKAR NATIONAL

INSTITUTE OF TECHNOLOGY, JALANDHAR

Week 3



NAME: Khushal Garg

ROLL NO.: 23103077

SECTION: 4A

GROUP: G3

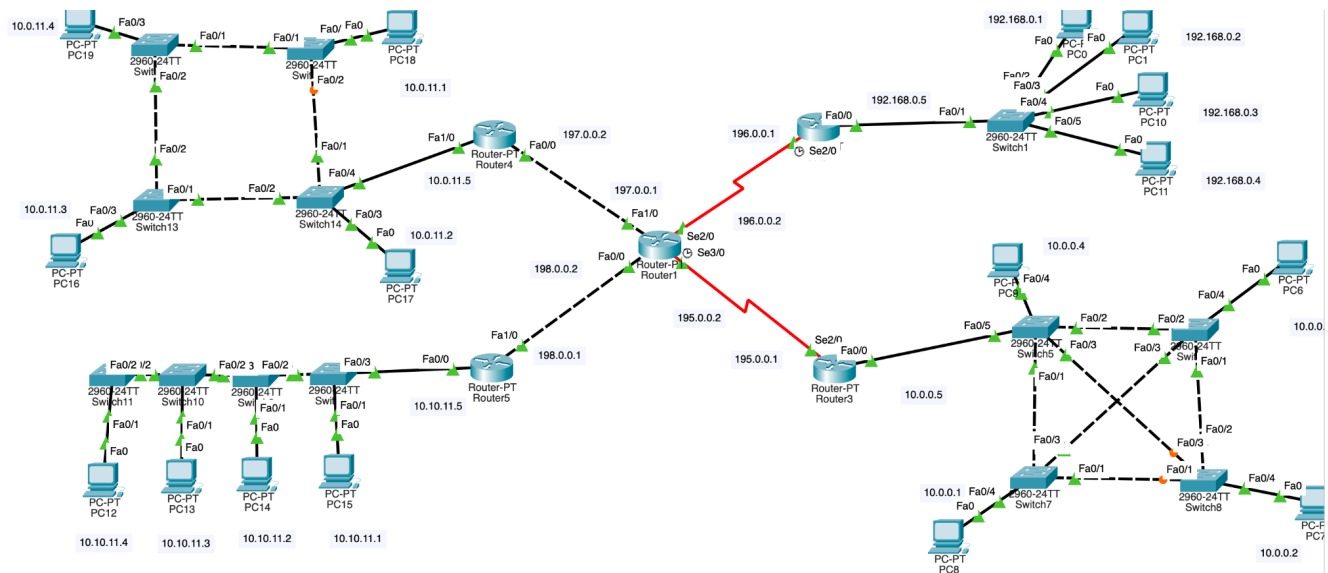
BRANCH: CSE

Objective 3: Create a scenario where four different departments (IT, HR, Sales, Finance) of different local area networks are directly connected through intermediate devices. Each group has at least 4 end systems. Each topology has at least one router, and each department is connected to a central router. Do the following for the given scenario:

- a) Provide the Static IP Address to every end device available in the network.
- b) Label each and every interface with their IP Address.
- c) Use a different topology for each department
- d) Configure static routing.
- e) Discuss the concept of the gateway and provide the default gateway address to devices.
- f) Show the data transmission between each department
- g) Also, explain the overall topology of the network.

1. Static IP Address Assignment

- Each end device in the network has been assigned a unique static IP address.
- The IP scheme follows a structured approach, with different subnets allocated to each department.
- Static IP addresses are :
 - IT Department (192.168.0.5/24):
 - PC1: 192.168.0.1
 - PC2: 192.168.0.2
 - PC3: 192.168.0.3
 - PC4: 192.168.0.4
 - HR Department (10.0.0.5/24):
 - PC1: 10.0.0.1
 - PC2: 10.0.0.2
 - PC3: 10.0.0.3
 - PC4: 10.0.0.4
 - Sales Department (10.10.11.5/24):
 - PC1: 10.10.11.1
 - PC2: 10.10.11.2
 - PC3: 10.10.11.3
 - PC4: 10.10.11.4
 - Finance Department (10.0.11.5/24):
 - PC1: 10.0.11.1
 - PC2: 10.0.11.2
 - PC3: 10.0.11.3
 - PC4: 10.0.11.4

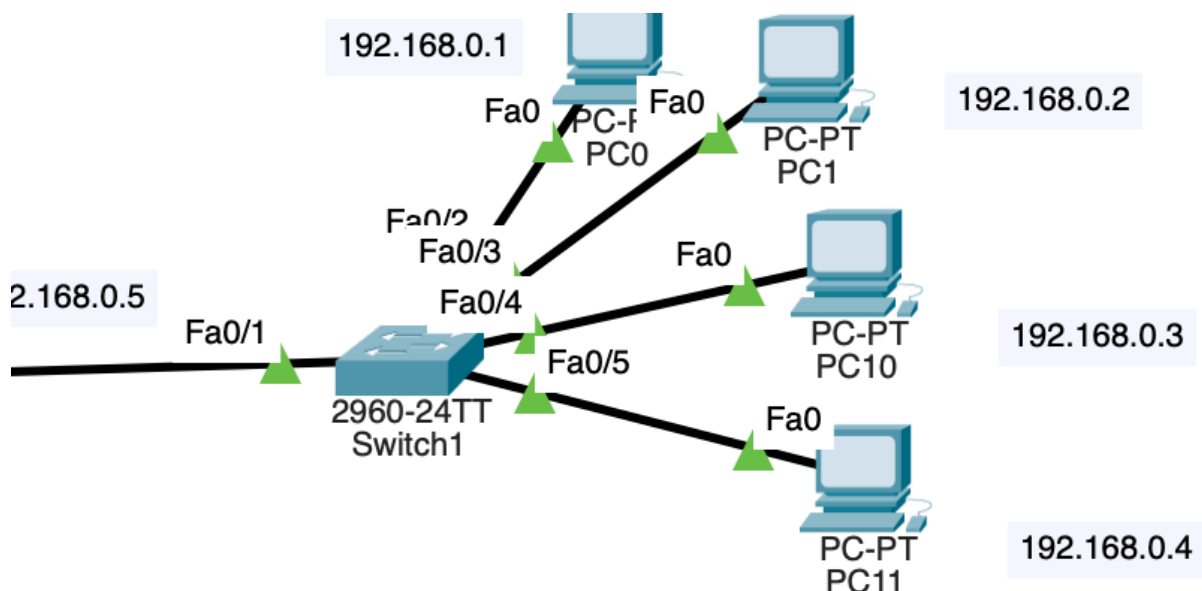


2. Interface Labeling

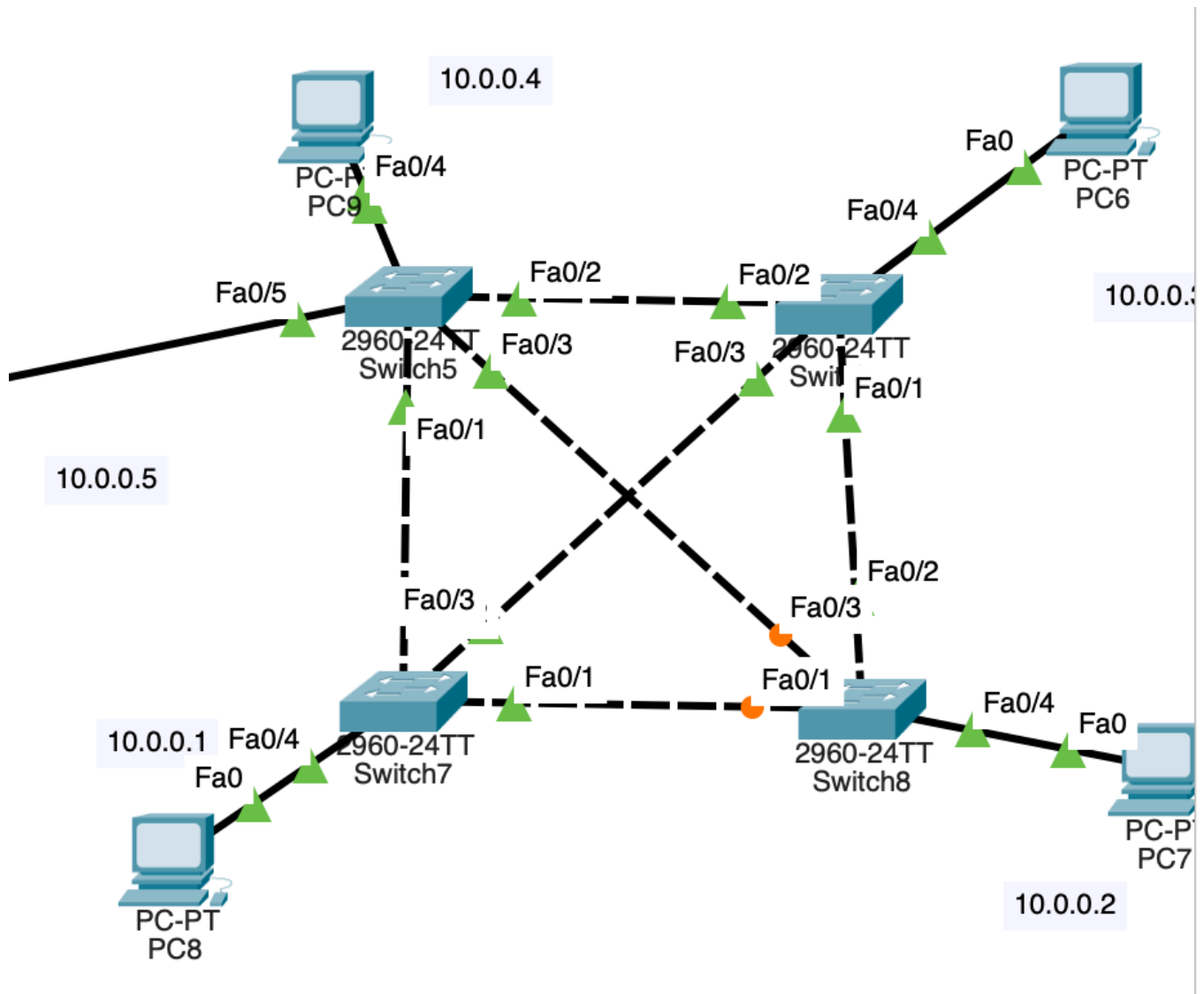
- All network interfaces, including routers and switches, have been labeled with their respective IP addresses to ensure clarity and ease of management.
- Example:
 - Router1 Fa0/0: 192.0.0.2
 - Router2 Fa0/0: 10.0.0.5
 - Router3 Fa0/0: 197.0.0.2

3. Different Topologies for Each Department

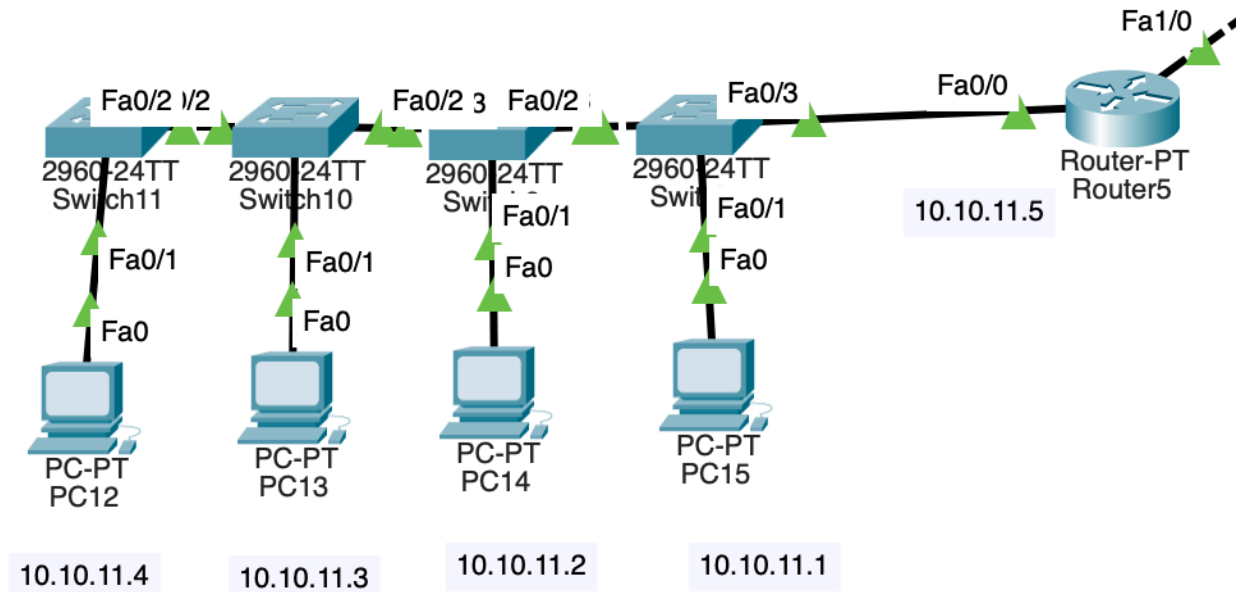
- IT Department: Star topology with a central switch connecting all devices



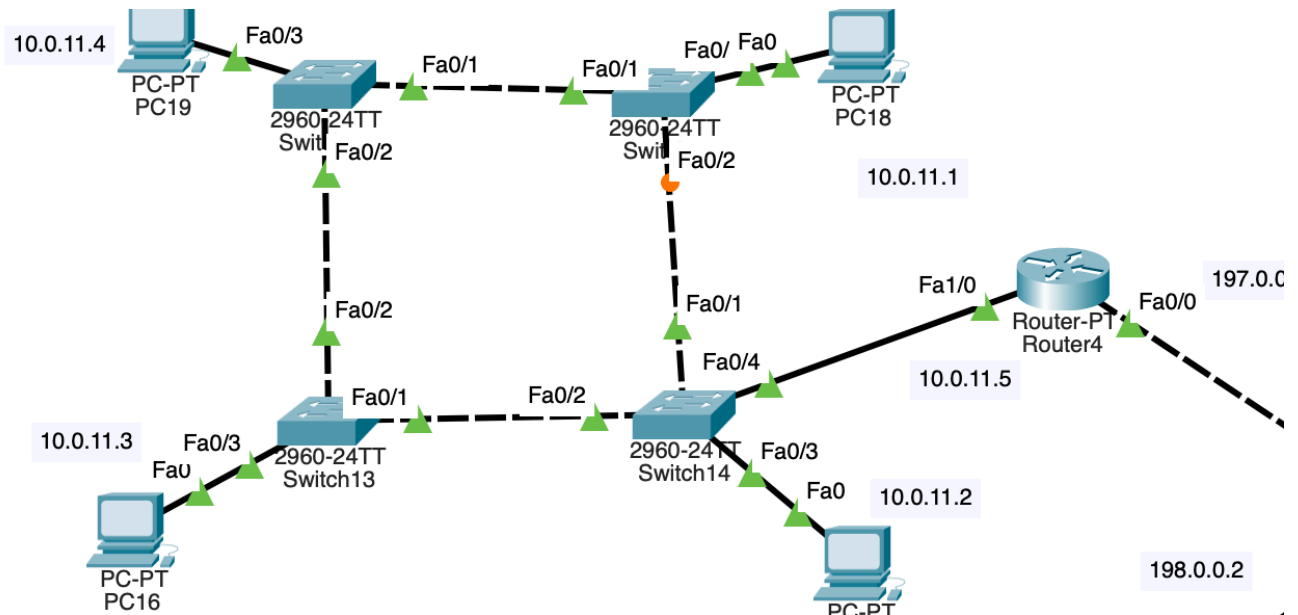
- HR Department: Mesh topology providing multiple paths for data transmission.



- Sales Department: Bus topology where devices share a common backbone.



- Finance Department: Ring topology ensuring redundancy in case of failure.



4. Static Routing Configuration

- Static routes have been configured on the routers to facilitate communication between departments.

Network Address
10.0.0.0/24 via 197.0.0.1
10.10.11.0/24 via 197.0.0.1

Network Address
10.0.0.0/24 via 197.0.0.1
10.10.11.0/24 via 197.0.0.1

Network Address
10.0.11.0/24 via 198.0.0.2

Network Address
10.0.0.0/24 via 197.0.0.1
10.10.11.0/24 via 197.0.0.1

5. Default Gateway Concept

- The default gateway is the router interface through which a device communicates outside its network.
- Each department has been assigned a gateway:
 - IT: 196.0.0.1
 - HR: 195.0.0.1
 - Sales: 198.0.0.1
 - Finance: 197.0.0.2

6. Data Transmission Between Departments

Fire	Last Status	Source	Destination	Type	Color	Time(sec)	Periodic	Num	Edit	Delete
	Successful	PC0	PC4	ICMP		0.000	N	0	(edit)	
	Successful	PC3(1)	PC0(1)	ICMP		0.000	N	1	(edit)	

7. Overall Network Topology Explanation

- A hierarchical network design was implemented, with a central router interconnecting departmental routers.
- Redundant links and different topologies were used to ensure network resilience.
- The network supports scalability and security by segmenting departments into different subnets.

DR. B.R. AMBEDKAR NATIONAL

INSTITUTE OF TECHNOLOGY, JALANDHAR

Week 4



NAME: Khushal Garg

ROLL NO.: 23103077

SECTION: 4A

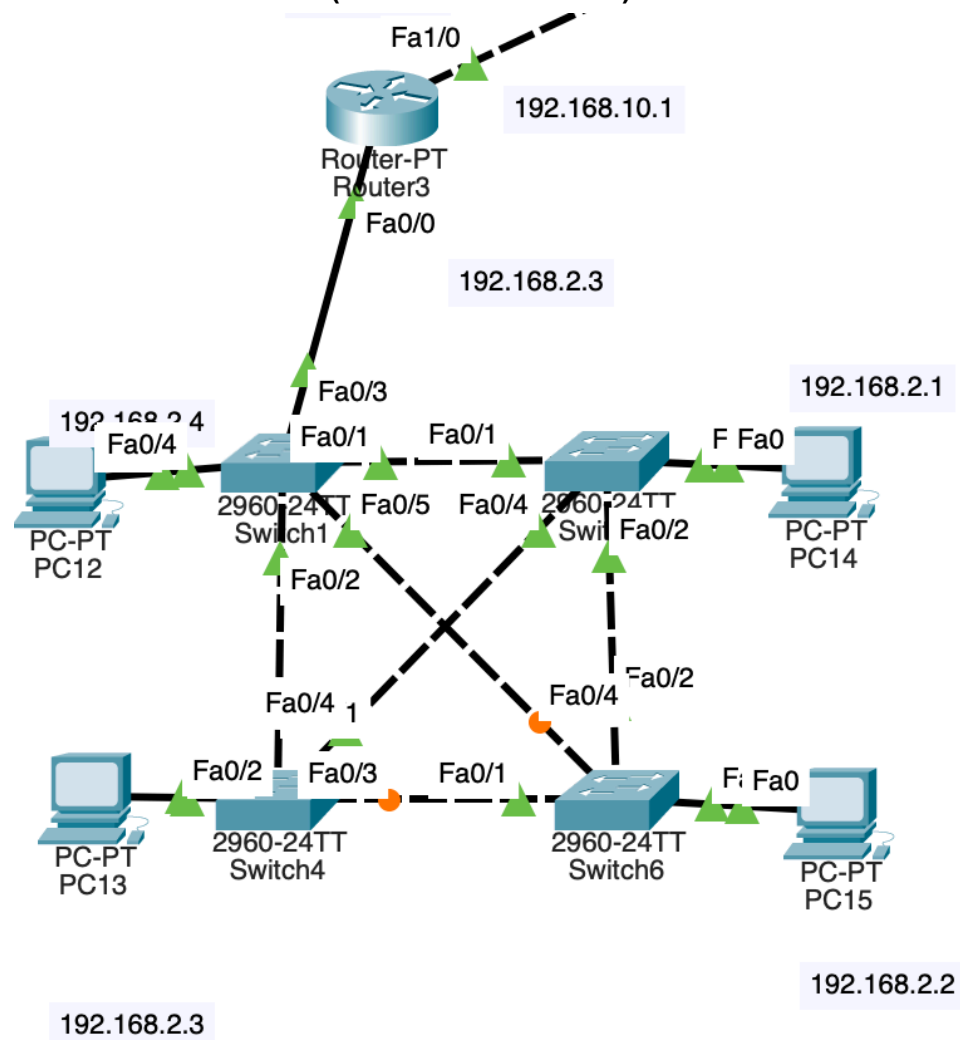
GROUP: G3

BRANCH: CSE

Objective : Create a scenario where four different departments (IT, HR, Sales, Finance) of different local area networks are directly connected through a central router. Each department has at least 4 end systems. Each topology has at least one router. Do the following for the given scenario:

- Provide the Static IP Address to every end device available in the network.
- Label each and every interface with their IP Address.
- Use a different topology for each department
- Configure RIP routing.
- Show the data transmission between each department.
- Also, explain the overall topology of the network.

1. HR DEPARTMENT (MESH TOPOLOGY):



Subnet: 192.168.2.0/24

Router (Router3): 192.168.1.2

End Devices:

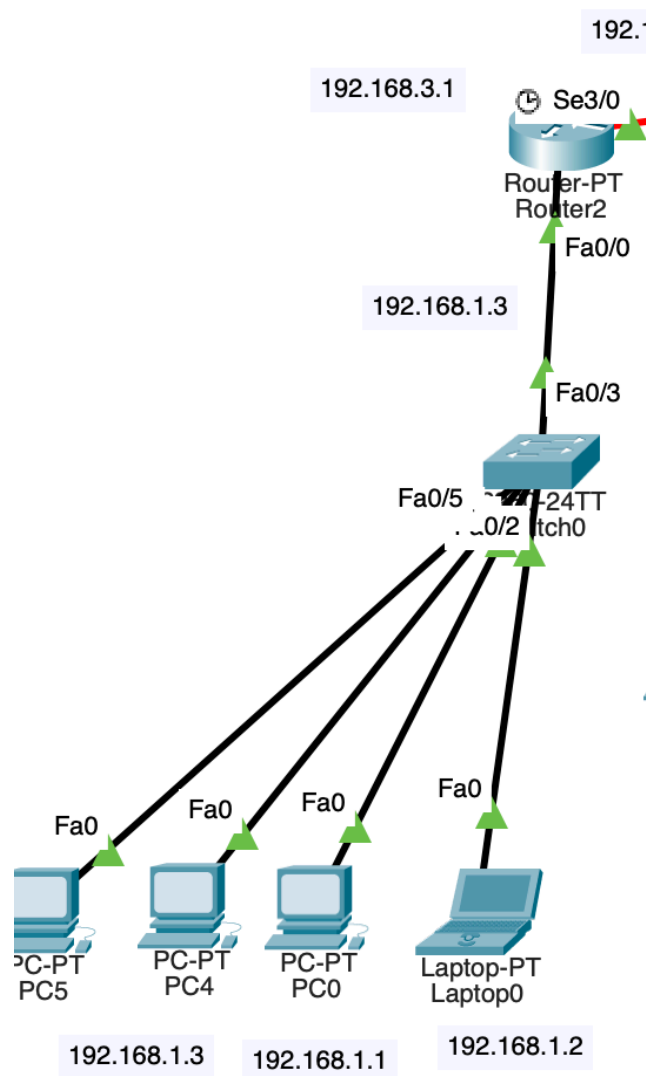
- PC12 - 192.168.2.4
- PC13 - 192.168.2.3
- PC14 - 192.168.2.1
- PC15 - 192.168.2.2

Switches: 4 interconnected switches forming a mesh topology.

Default Gateway: 192.168.1.2

Connection to Router 1: 192.168.10.1 -> 192.168.10.2

2. FINANCE DEPARTMENT (STAR TOPOLOGY):



Subnet: 192.168.1.0/24

Router (Router1): 192.168.5.1

End Devices:

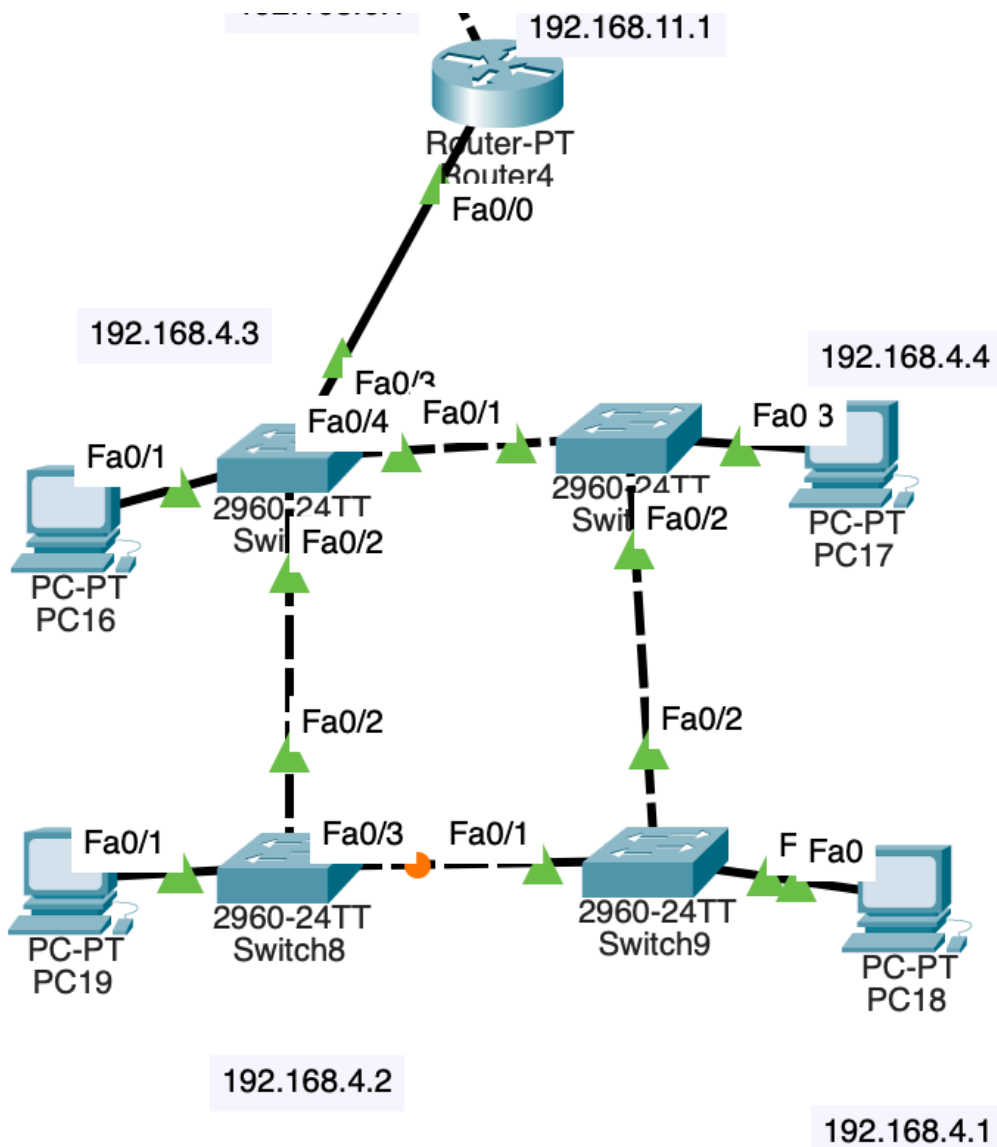
- PC0 - 192.168.1.1
- PC4 - 192.168.1.3
- PC5 - 192.168.1.4
- Laptop-PT - 192.168.1.2

Switches: A single switch connecting all PCs in **star topology**.

Default Gateway: 192.168.5.1

Connection to Router 4: 192.168.9.1 -> 192.168.9.2

3.IT DEPARTMENT (RING TOPOLOGY):



Subnet: 192.168.4.0/24

Router (Router2): 192.168.7.1

End Devices:

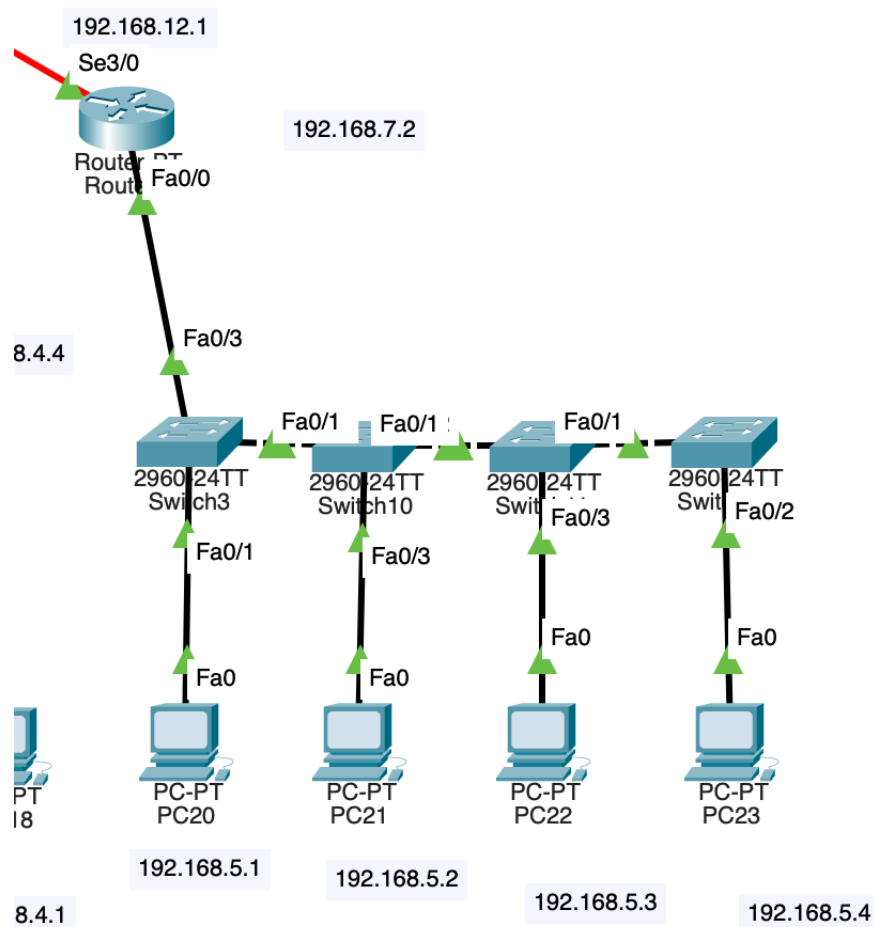
- PC16 - 192.168.4.3
- PC17 - 192.168.4.4
- PC18 - 192.168.4.1
- PC19 - 192.168.4.2

Switches: 4 interconnected switches forming a ring topology.

Default Gateway: 192.168.7.1

Connection to Router 4: 192.168.11.1 -> 192.168.11.2

4.SALES DEPARTMENT (BUS TOPOLOGY):



Subnet: 192.168.5.0/24

Router (Router 3): 192.168.8.1

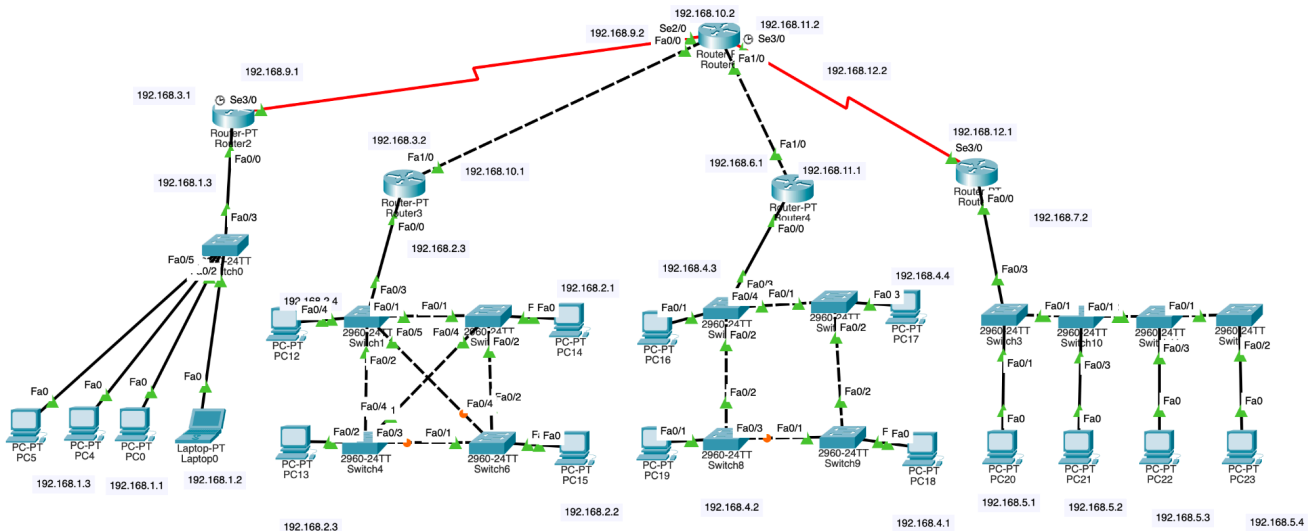
End Devices:

- PC20 - 192.168.5.1
- PC21 - 192.168.5.2
- PC22 - 192.168.5.3
- PC23 - 192.168.5.4

Switches: A series of switches connected linearly, forming a **bus topology**.

Default Gateway: 192.168.8.1

Connection to Router 4: 192.168.12.1 -> 192.168.12.2



1. **RIP Routing is used** to enable dynamic connectivity between departments.
2. **A central router (Router4) interconnects all department routers.**
3. **Each department has its own subnet and router** to isolate traffic while allowing communication when needed.
4. **Data transmission is managed dynamically through RIP.**

DR. B.R. AMBEDKAR NATIONAL
INSTITUTE OF TECHNOLOGY, JALANDHAR

Week 5



NAME: Khushal Garg

ROLL NO.: 23103077

SECTION: 4A

GROUP: G3

BRANCH: CSE

Objective#5: Create a scenario where four different departments (IT, HR, Sales, Finance) of different local area networks are directly connected through a central router. Each department has at least 4 end systems. Each topology has at least one router. Do the following for the given scenario:

- a) Provide the Static IP Address to every end device available in the network.
- b) Label each and every interface with their IP Address.
- c) Use a different topology for each department
- d) Configure OSPF routing.
- e) Show the data transmission between each department
- f) Also, explain the overall topology of the network.

(a) Static IP Addressing:

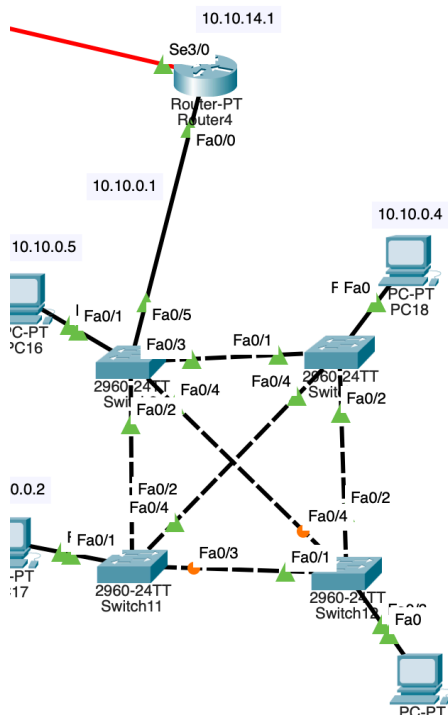
- **Centralized Routing:** A main router connects all departments, ensuring seamless communication.
- **Departmental Network Topologies:**
 - **IT:** Mesh Topology
 - **HR:** Bus Topology
 - **Sales:** Ring Topology
 - **Finance:** Star Topology
- **Subnet Allocation:**
 - **IT:** 10.10.0.0/24
 - **HR:** 192.168.1.0/24
 - **Sales:** 10.0.0.0/24
 - **Finance:** 192.168.0.0/24
- **Static IP Assignment:**
 - Every device is assigned a fixed IP within its department's subnet.
 - Router interfaces are given unique static IPs for better network management.

(b) Label Interfaces

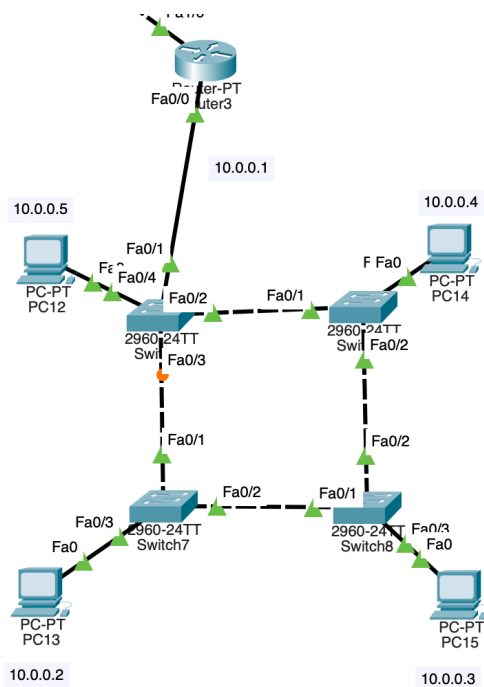
- **Router Interface Example:**
 - **Fa0/0 (IT) → 10.10.0.1**
 - **Fa0/1 (HR) → 192.168.1.1**
 - **Fa0/2 (Sales) → 10.0.0.1**
 - **Fa0/3 (Finance) → 192.168.0.1**

(c) Topologies :

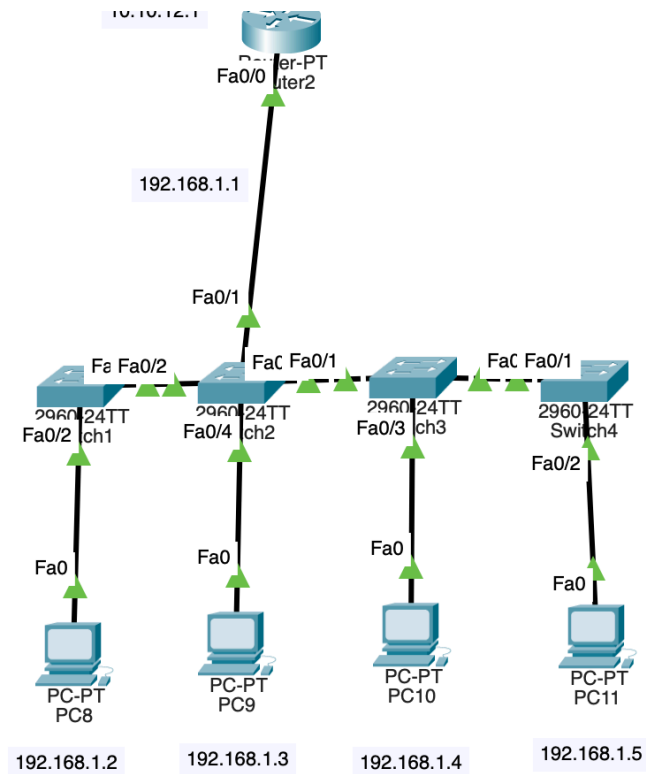
1. **Mesh Topology (HR Department):** Offers multiple redundant connections, enhancing reliability and fault tolerance.



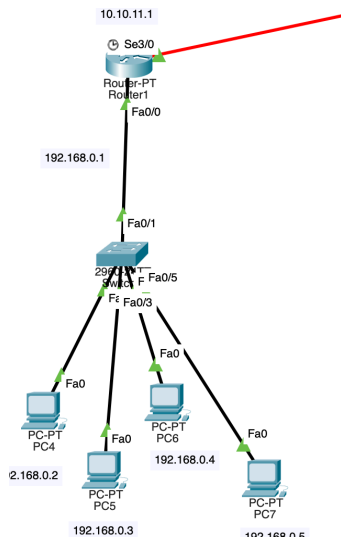
2. Ring Topology (IT Department) - Creates a continuous loop where data travels in a circular path, ensuring efficient communication.



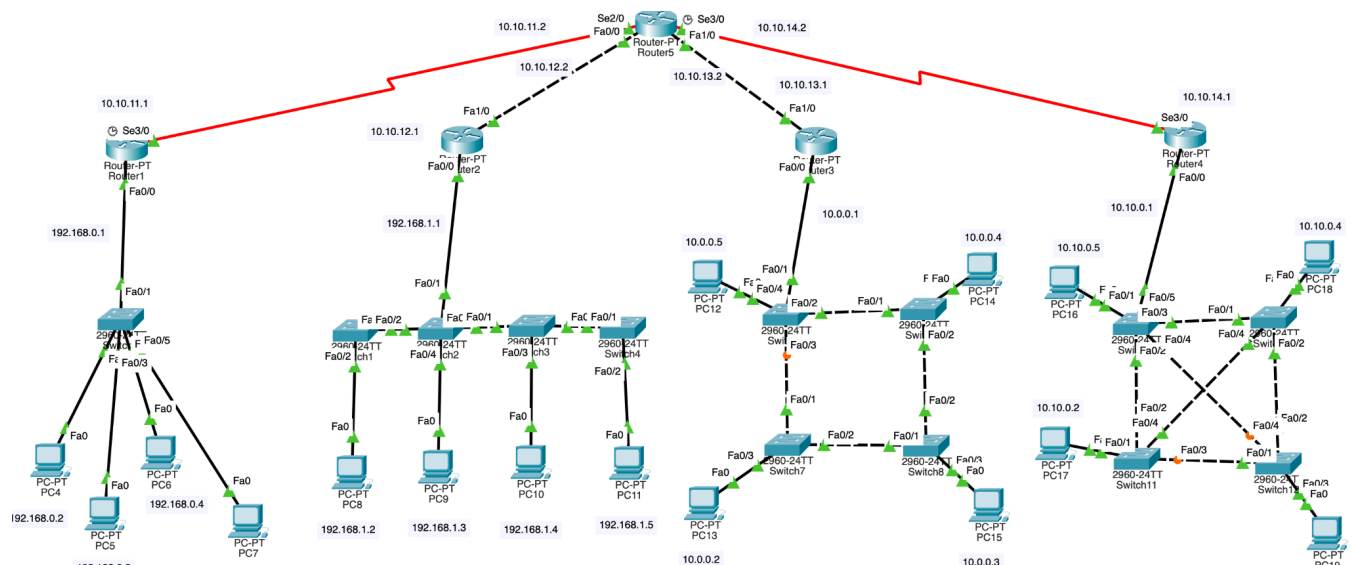
3. Bus Topology (Sales Department) - All devices share a single central backbone, making communication simple and efficient.



4. Star Topology (Finance Department) - A central switch links all devices, ensuring efficient communication and easy management.



Overall Network:



d) Configure OSPF

1. Access each router's CLI and enable OSPF.

(e) Show Data Transmission

```
Router(config)# router ospf 1
Router(config-router)# network 192.168.0.0 0.0.0.255 area 0
Router(config-router)# network 192.168.1.0 0.0.0.255 area 0
Router(config-router)# network 10.0.0.0 0.0.0.255 area 0
Router(config-router)# network 192.168.3.0 0.0.0.255 area 0
```

f) Topology Explanation

Each router is configured with OSPF (Open Shortest Path First) as the dynamic routing protocol, allowing seamless communication between different networks. The network follows a hierarchical structure, with each department operating within its own subnet, managed by a dedicated departmental router. Router 4 serves as the central hub for inter-departmental communication.

This setup provides:

- **Scalability, as OSPF dynamically updates routing tables.**
- **Redundancy, supported by the HR department's mesh topology.**
- **Efficiency, ensured by the star topology used in Finance.**
- **Simple expansion, facilitated by the bus topology in Sales.**

DR. B.R. AMBEDKAR NATIONAL
INSTITUTE OF TECHNOLOGY, JALANDHAR

Week 6



NAME: Khushal Garg

ROLL NO.: 23103077

SECTION: 4A

GROUP: G3

BRANCH: CSE

Objective#6: Write a menu-driven program to implement and validate the following error detection/correction techniques:

- a) Checksum
- b) Longitudinal Redundancy Check
- c) Cyclic redundancy check
- d) Hamming Code

```
import numpy as np

def checksum(data, block_size):
    blocks = [data[i:i+block_size] for i in range(0, len(data), block_size)]
    total = sum(int(block, 2) for block in blocks)
    while total >> block_size:
        total = (total & ((1 << block_size) - 1)) + (total >> block_size)
    checksum = (~total) & ((1 << block_size) - 1)
    return format(checksum, f'0{block_size}b')

def lrc(data):
    rows = data.split()
    parity_bits = ''
    for i in range(len(rows[0])):
        column_bits = [row[i] for row in rows]
        parity_bits += str(column_bits.count('1') % 2)
    return parity_bits

def crc(data, polynomial):
    data += '0' * (len(polynomial) - 1)
    data = list(data)
    poly = list(polynomial)

    for i in range(len(data) - len(poly) + 1):
        if data[i] == '1':
            for j in range(len(poly)):
                data[i + j] = str(int(data[i + j]) ^ int(poly[j]))
    return ''.join(data[-(len(poly) - 1):])

def hamming_code(data):
    n = len(data)
    r = 0

    while (2**r) < (n + r + 1):
        r += 1
```

```

code = ['0'] * (n + r)

j = 0
for i in range(len(code)):
    if (i + 1) & (i) == 0:
        continue
    code[i] = data[j]
    j += 1

for i in range(r):
    index = 2**i - 1
    parity = 0

    for j in range(index, len(code), 2 * (index + 1)):
        for k in range(j, min(j + index + 1, len(code))):
            parity ^= int(code[k])

    code[index] = str(parity)

return ''.join(code)

def main():
    while True:
        print("\nMenu:")
        print("1. Checksum")
        print("2. Longitudinal Redundancy Check (LRC)")
        print("3. Cyclic Redundancy Check (CRC)")
        print("4. Hamming Code")
        print("5. Exit")
        choice = input("Enter your choice: ")

        if choice == '1':
            data = input("Enter binary data: ")
            block_size = int(input("Enter block size: "))
            print("Checksum: ", checksum(data, block_size))
        elif choice == '2':
            data = input("Enter binary data (space-separated rows): ")
            print("LRC: ", lrc(data))
        elif choice == '3':
            data = input("Enter binary data: ")
            polynomial = input("Enter CRC polynomial: ")
            print("CRC Checksum: ", crc(data, polynomial))
        elif choice == '4':

```

```

        data = list(input("Enter binary data: "))
        print("Hamming Code: ", hamming_code(data))
    elif choice == '5':
        break
    else:
        print("Invalid choice. Try again.")

if __name__ == "__main__":
    main()

```

Error Detection and Correction Techniques

Error detection and correction are essential techniques in computer networks and digital communication to ensure reliable data transmission. Errors can occur due to noise, interference, or hardware faults, leading to corrupted data. To resolve such issues, various methods are used to detect and correct errors in transmitted messages.

a) Checksum

The **Checksum** is an error-detection method that works by summing the data units in a message and appending the sum to the message as a checksum value. At the receiver end, the same summation is performed, and if the computed checksum does not match the transmitted checksum, an error is detected. **Steps:**

Sender Side:

1. Take input for **N data words** and divide them into **fixed-sized blocks**.
2. Add all blocks using **binary addition** to compute the sum.
3. Take the **complement of the sum** to generate the checksum.
4. Transmit the **message along with the checksum**.

Receiver Side:

1. Add all received **data blocks**, including the **checksum**, using **binary addition**.
2. If the sum results in **all ones** (complemented zero), no error is detected; otherwise, an **error is present**.

The screenshot shows a terminal window with the following content:

```

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
khushalgarg@Khushals-MacBook-Air Python % /usr/local/bin/python3 /Users/khushalgarg/Desktop/Python/CN_LAB/Lab_6/lab6.py

Menu:
1. Checksum
2. Longitudinal Redundancy Check (LRC)
3. Cyclic Redundancy Check (CRC)
4. Hamming Code
5. Exit
Enter your choice: 1
Enter binary data: 110101110010
Enter block size: 4
Checksum: 1000

Menu:
1. Checksum
2. Longitudinal Redundancy Check (LRC)
3. Cyclic Redundancy Check (CRC)
4. Hamming Code
5. Exit
Enter your choice: 5
khushalgarg@Khushals-MacBook-Air Python %

```

2. Longitudinal Redundancy Check (LRC)

LRC is a block-based error detection method where parity bits are added for each data word, and an additional **longitudinal parity** row is computed for all data bits in a block. **Steps:**

Sender Side:

1. Take input for the **number of rows and columns** of data.
2. Arrange the data in a **matrix format**, with each row representing one data word.
3. Compute the **parity bit for each column** using **XOR** and append it as an **extra row**(LRC parity row).
4. Transmit the **data along with the LRC row**.

Receiver Side:

1. Recompute the **LRC row** using the received data.
2. Compare the **computed parity row** with the received **LRC row**.
3. If they **match**, the data is correct; otherwise, an **error is detected**.

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

/usr/local/bin/python3 /Users/khushalgarg/Desktop/Python/CN_LAB/Lab_6/lab6.py
khushalgarg@Khushals-MacBook-Air Python % /usr/local/bin/python3 /Users/khushalgarg/Desktop/Python/CN_LAB/Lab_6/lab6.py

Menu:
1. Checksum
2. Longitudinal Redundancy Check (LRC)
3. Cyclic Redundancy Check (CRC)
4. Hamming Code
5. Exit
Enter your choice: 2
Enter binary data (space-separated rows): 1101 0110 1010
LRC: 0001

Menu:
1. Checksum
2. Longitudinal Redundancy Check (LRC)
3. Cyclic Redundancy Check (CRC)
4. Hamming Code
5. Exit
Enter your choice: 5
khushalgarg@Khushals-MacBook-Air Python %
```

3. Cyclic Redundancy Check (CRC)

CRC is an efficient method for error detection that uses polynomial division to compute a checksum known as the **CRC code**. The sender appends the CRC code to the message, and the receiver verifies it by performing division again. **Steps:**

Sender Side:

1. Take input for **data bits** and a **generator polynomial** (e.g., $x^3+x+1x^3 + x + 1x^3+x+1$).
2. Append **n zero bits** to the data, where **n** is the degree of the polynomial minus 1.
3. Perform **binary division (XOR-based)** between the **augmented data** and the **generator polynomial**.
4. The **remainder** from the division is the **CRC checksum**.
5. Append the **CRC checksum** to the data and **transmit**.

Receiver Side:

1. Perform **binary division** on the **received data** using the **same generator polynomial**.
2. If the **remainder is all zeros**, no error is detected; otherwise, an **error is present**.

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
khushalgarg@Khushals-MacBook-Air Python % /usr/local/bin/python3 /Users/khushalgarg/Desktop/Python/CN_LAB/Lab_6/lab6.py

Menu:
1. Checksum
2. Longitudinal Redundancy Check (LRC)
3. Cyclic Redundancy Check (CRC)
4. Hamming Code
5. Exit
Enter your choice: 3
Enter binary data: 11010011101100
Enter CRC polynomial: 1011
CRC Checksum: 100

Menu:
1. Checksum
2. Longitudinal Redundancy Check (LRC)
3. Cyclic Redundancy Check (CRC)
4. Hamming Code
5. Exit
Enter your choice: 5
khushalgarg@Khushals-MacBook-Air Python %
```

4. Hamming Code (Error Detection & Correction)

The **Hamming Code** is an error-detection and error-correction technique that uses redundant bits placed at specific positions in the data to detect and correct single-bit errors. **Steps:**

Sender Side:

1. Determine the number of **redundant bits (R)** required using the formula:
 $2^R \geq m + r + 1$, where **m** is the number of **data bits** and **R** is the number of **parity bits**.
2. Insert **redundant bits** at positions that are **powers of two** (1, 2, 4, 8, ...).
3. Compute **parity bits** using **XOR operations** on specific groups of **data bits**.
4. Transmit the **message with parity bits**.

Receiver Side:

1. **Recalculate parity bits** using the same XOR-based method.
2. If **no error** is detected, the message is correct.
3. If an **error** is found, determine the **error bit position** using the calculated parity values and **correct the bit**.

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Python: lab6
khushalgarg@Khushals-MacBook-Air Python % /usr/local/bin/python3 /Users/khushalgarg/Desktop/Python/CN_LAB/Lab_6/lab6.py

Menu:
1. Checksum
2. Longitudinal Redundancy Check (LRC)
3. Cyclic Redundancy Check (CRC)
4. Hamming Code
5. Exit
Enter your choice: 4
Enter binary data: 1011
Hamming Code: 0110011

Menu:
1. Checksum
2. Longitudinal Redundancy Check (LRC)
3. Cyclic Redundancy Check (CRC)
4. Hamming Code
5. Exit
Enter your choice: 5
khushalgarg@Khushals-MacBook-Air Python %
```

DR. B.R. AMBEDKAR NATIONAL
INSTITUTE OF TECHNOLOGY, JALANDHAR

Week 7



NAME: Khushal Garg

ROLL NO.: 23103077

SECTION: 4A

GROUP: G3

BRANCH: CSE

Introduction :

Bit stuffing and byte stuffing are **data framing techniques** used in network communication to prevent special control characters from being misinterpreted as actual data. **Bit stuffing** is used in **bit-oriented protocols** like **HDLC**, where a **0** is inserted after every sequence of five consecutive **1**s to avoid confusion with frame delimiters. On the other hand, **byte stuffing** is used in **byte-oriented protocols** like **PPP**, where a special escape character (**ESC**) is added before control characters (e.g., **FLAG**) to distinguish them from normal data. Both techniques ensure **data integrity** and **proper synchronization** in transmission, preventing errors due to unintended delimiter detection.

Techniques Overview :

(a) Byte Stuffing (Character Stuffing) :-

Byte stuffing is a **data framing technique** used in **byte-oriented protocols** (like **PPP**) to ensure that special control characters (such as **FLAG**) do not interfere with data transmission. It involves inserting an **escape character (ESC)** before any special characters in the data.

Client Side (Sender) implementation :-

1. Create a socket to connect to the server.
2. Take user input (string that may contain special characters like **F** and **E**).
3. Apply byte stuffing:
 - Replace every E with EE
 - Replace every F with EF
4. Send the stuffed data to the server over the socket.
5. Close the connection.

Server Side (Receiver) implementation :-

1. Create a server socket and listen for client connections.
2. Accept a client request when a connection is established.
3. Receive the stuffed data from the client.
4. Apply byte unstuffing:
 - Replace every EF with F (restore original FLAG)
 - Replace every EE with E (restore original ESC).
5. Display the unstuffed message.
6. Close the connection.

Server Side :-

```
import socket

FLAG = 'F'

ESC = 'E'

def byte_unstuffing(data):

    return data.replace(ESC + FLAG, FLAG).replace(ESC + ESC, ESC)

server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

server.bind(('localhost', 12346))

server.listen(1)

print("Server listening on port 12346")

conn, addr = server.accept()

print(f"Connected to {addr}")

data = conn.recv(1024).decode()

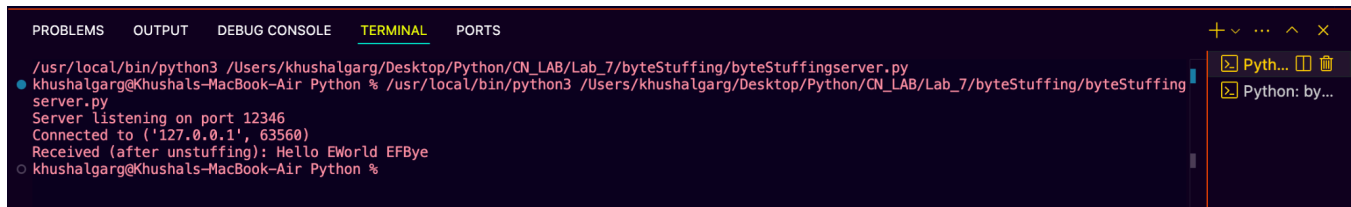
unstuffed_data = byte_unstuffing(data)

print("Received (after unstuffing):", unstuffed_data)

conn.close()

server.close()
```

Output :-



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
/usr/local/bin/python3 /Users/khushalgarg/Desktop/Python/CN_LAB/Lab_7/byteStuffing/byteStuffingserver.py
khushalgarg@Khushals-MacBook-Air Python % /usr/local/bin/python3 /Users/khushalgarg/Desktop/Python/CN_LAB/Lab_7/byteStuffing/byteStuffingserver.py
Server listening on port 12346
Connected to ('127.0.0.1', 63560)
Received (after unstuffing): Hello EWorld EFBye
khushalgarg@Khushals-MacBook-Air Python %
```

Client side :-

```
import socket

FLAG = 'F'

ESC = 'E'

def byte_unstuffing(data):

    return data.replace(ESC + FLAG, FLAG).replace(ESC + ESC, ESC)

client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

client.connect(('localhost', 12346))

data = input("Enter message with FLAG/ESC: ")

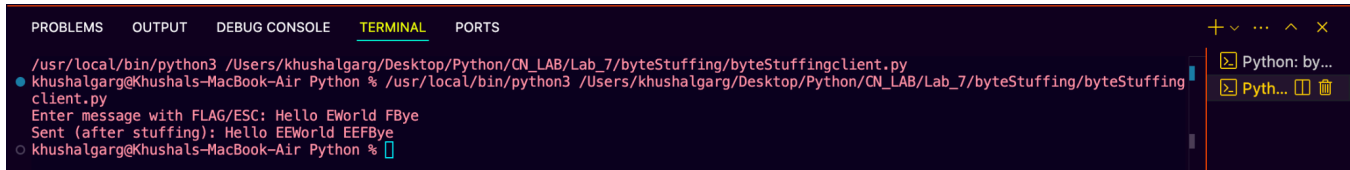
stuffed_data = data.replace(FLAG, ESC + FLAG).replace(ESC, ESC + ESC)

client.send(stuffed_data.encode())

print("Sent (after stuffing):", stuffed_data)

client.close()
```

Output :-



```
/usr/local/bin/python3 /Users/khushalgarg/Desktop/Python/CN_LAB/Lab_7/byteStuffing/byteStuffingclient.py
khushalgarg@Khushals-MacBook-Air Python % /usr/local/bin/python3 /Users/khushalgarg/Desktop/Python/CN_LAB/Lab_7/byteStuffing/byteStuffingclient.py
Enter message with FLAG/ESC: Hello EWorld FByte
Sent (after stuffing): Hello EEWorld EEFFByte
khushalgarg@Khushals-MacBook-Air Python %
```

(b) Bit Stuffing :-

Bit stuffing is a **data framing technique** used in **bit-oriented protocols** like **HDLC** to prevent the accidental misinterpretation of data as control information. When a sender detects **five consecutive ‘1’ bits**, it **inserts a ‘0’** to ensure that the data does not mimic the frame delimiter pattern.

Client Side (Sender) implementation :-

1. **Create a socket** to connect to the server.
2. **Take user input** (binary string).
3. Apply bit stuffing:
Insert a **0** after any sequence of **five consecutive ‘1’s**.
4. **Send the stuffed binary data** to the server.
5. **Close the connection**.

Server Side (Receiver) implementation :-

1. **Create a server socket** and listen for client connections.
2. **Accept a client request** when a connection is established.
3. **Receive the stuffed binary data** from the client.
4. Apply bit unstuffing:
Remove any **0** that appears **after five consecutive ‘1’s**.
5. **Display the unstuffed binary data**.
6. **Close the connection**.

Server side :-

```
import socket

def bit_stuffing(data):

    stuffed = ''

    count = 0

    for bit in data:

        if bit == '1':

            count += 1

        else:

            count = 0

            stuffed += bit

        if count == 5:

            stuffed += '0'

            count = 0

    return stuffed

server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

server.bind(('localhost', 12345))

server.listen(1)

print("Server listening on port 12345")

conn, addr = server.accept()

print(f"Connected to {addr}")
```

```
data = conn.recv(1024).decode()

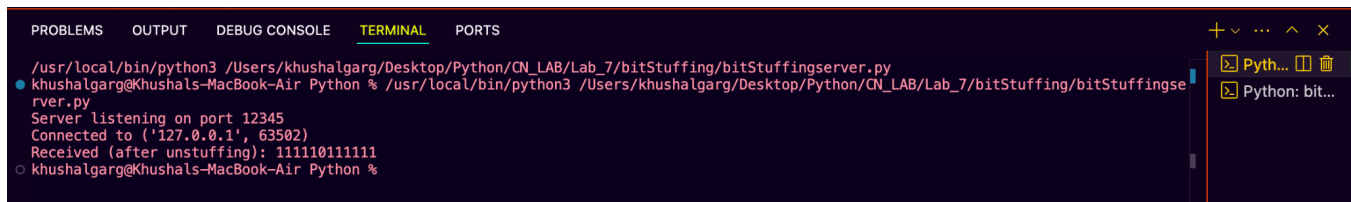
unstuffed_data = data.replace('111110', '11111')

print("Received (after unstuffing):", unstuffed_data)

conn.close()

server.close()
```

Output :-



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
/usr/local/bin/python3 /Users/khushalgarg/Desktop/Python/CN_LAB/Lab_7/bitStuffing/bitStuffingserver.py
khushalgarg@Khushals-MacBook-Air Python % /usr/local/bin/python3 /Users/khushalgarg/Desktop/Python/CN_LAB/Lab_7/bitStuffing/bitStuffingserver.py
Server listening on port 12345
Connected to ('127.0.0.1', 63502)
Received (after unstuffing): 1111011111
khushalgarg@Khushals-MacBook-Air Python %
```

Client Side :-

```
import socket

def bit_stuffing(data):

    stuffed = ''

    count = 0

    for bit in data:

        if bit == '1':

            count += 1

        else:

            count = 0

    stuffed += bit
```

```

        if count == 5:

            stuffed += '0'

            count = 0

        return stuffed

client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

client.connect(('localhost', 12345))


data = input("Enter binary data: ")

stuffed_data = bit_stuffing(data)

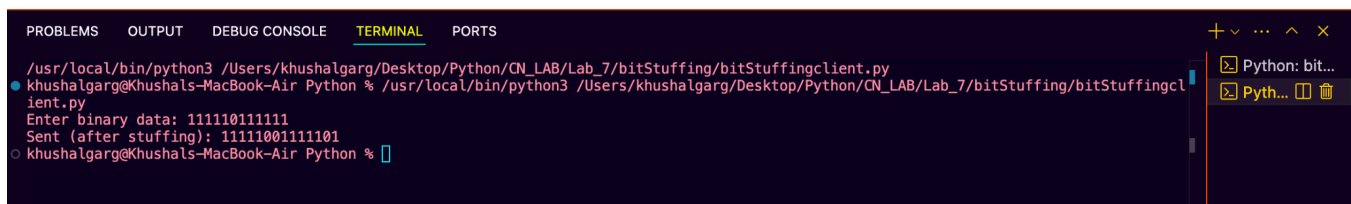
client.send(stuffed_data.encode())

print("Sent (after stuffing):", stuffed_data)


client.close()

```

Output :-



```

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
/usr/local/bin/python3 /Users/khushalgarg/Desktop/Python/CN_LAB/Lab_7/bitStuffing/bitStuffingclient.py
khushalgarg@Khushals-MacBook-Air Python % /usr/local/bin/python3 /Users/khushalgarg/Desktop/Python/CN_LAB/Lab_7/bitStuffing/bitStuffingcl
ient.py
Enter binary data: 1111011111
Sent (after stuffing): 111100111101
khushalgarg@Khushals-MacBook-Air Python %

```

DR. B.R. AMBEDKAR NATIONAL
INSTITUTE OF TECHNOLOGY, JALANDHAR

Week 8



NAME: Khushal Garg

ROLL NO.: 23103077

SECTION: 4A

GROUP: G3

BRANCH: CSE

